BigQuery Partitioning & Clustering

A Hands-On Demo Walkthrough — Day → Month → Year → Timestamp → Ingestion-Time → Clustering → Partition+Clustering

---

This guide uses one synthetic e-commerce **orders** dataset (order_id, order_date, order_ts, customer_id, country, category, amount) to demonstrate every BigQuery partitioning and clustering strategy, in the order you'd present them in a training session. Each section gives the DDL, what it does, and — most importantly — *why* you'd choose it and how it changes query cost and performance.

Prerequisite: the base table `partition_demo.orders_raw` should already exist (built from the provided orders_sample.csv or generated in BigQuery). Every table below is built with `CREATE OR REPLACE TABLE ... AS SELECT * FROM orders_raw`, so the row-level data is identical across every variant — only the physical storage layout changes.

Time-Unit Column Partitioning — DAY

What it does: splits the table into one physical partition per calendar day, based on the `order_date` column.

```
CREATE OR REPLACE TABLE `partition_demo.orders_by_day`
PARTITION BY order_date
CLUSTER BY country, category
AS SELECT * FROM `partition_demo.orders_raw`;
```

Why DAY, and why here first

order_date is a native DATE column driving day-level reporting — the textbook case for time-unit partitioning, so it's the natural starting point. When a query filters `WHERE order_date BETWEEN ...`, BigQuery reads only the partitions (days) that overlap the range and skips everything else on disk. That's the entire mechanism: less data read from storage means fewer bytes billed and a faster query.

Granularity matters: BigQuery allows at most 4,000 partitions per table. Our data spans ~1,460 days (4 years), so DAY partitioning is safe here — but with 10+ years of history, DAY partitioning risks hitting that ceiling, which is exactly why MONTH and YEAR exist as alternatives.

Verify pruning

```
SELECT country, SUM(amount) AS revenue
FROM `partition_demo.orders_by_day`
WHERE order_date BETWEEN '2023-06-01' AND '2023-06-30'
GROUP BY country;

-- Compare against orders_raw (no partitioning) using the same filter
-- and check "bytes processed" in the console before running each.
```

Run the same filter against orders_raw and orders_by_day, and check the estimated bytes shown by the console before clicking Run. orders_by_day should show a small fraction of the bytes that orders_raw scans, because only ~30 daily partitions are touched instead of the whole table.

Time-Unit Column Partitioning — MONTH

What it does: groups rows into one partition per calendar month using `DATE_TRUNC(order_date, MONTH)`.

```
CREATE OR REPLACE TABLE `partition_demo.orders_by_month`
PARTITION BY DATE_TRUNC(order_date, MONTH)
CLUSTER BY customer_id
AS SELECT * FROM `partition_demo.orders_raw`;
```

Why MONTH

This is the answer to DAY partitioning's partition-count problem. If a table holds many years of history and is mostly queried at monthly or quarterly grain (financial close, monthly cohort reports), DAY partitioning creates far more partitions than the query patterns need, adding metadata overhead without a matching benefit. MONTH keeps the partition count low (48 months over 4 years here, versus ~1,460 days) while still pruning effectively for the queries that matter.

Clustering by customer_id (an integer, high-cardinality column) here is a deliberate contrast point: clustering keys aren't limited to strings, and a good clustering key is simply one that's frequently filtered or joined on, regardless of type.

Query

```
SELECT COUNT(*) AS order_count
FROM `partition_demo.orders_by_month`
WHERE order_date BETWEEN '2023-06-01' AND '2023-06-30';

-- Explicit truncated-column form (makes the pruning logic obvious to learners)
SELECT COUNT(*) AS order_count
FROM `partition_demo.orders_by_month`
WHERE DATE_TRUNC(order_date, MONTH) = DATE_TRUNC(DATE '2023-06-15', MONTH);
```

Time-Unit Column Partitioning — YEAR

What it does: the coarsest time-unit partitioning — one partition per calendar year.

```
CREATE OR REPLACE TABLE `partition_demo.orders_by_year`
PARTITION BY DATE_TRUNC(order_date, YEAR)
AS SELECT * FROM `partition_demo.orders_raw`;
```

Why YEAR

Suited to archival, compliance, or trend-analysis tables queried at yearly grain — think regulatory retention tables or year-over-year dashboards where nobody ever asks for a single day or month. The trade-off is the mirror image of DAY partitioning: fewer, larger partitions mean a query that only needs one month out of a year still reads the entire year's data, so pruning granularity is coarser and query cost is higher for narrow date filters.

This is the best table to pair with orders_by_day for a live contrast: filter both to a single month and compare bytes processed. orders_by_year will scan roughly 12x more data than needed for that filter, while orders_by_day scans almost exactly what's needed.

Query

```
SELECT COUNT(*) AS order_count
FROM `partition_demo.orders_by_year`
WHERE order_date BETWEEN '2023-01-01' AND '2023-12-31';

-- Multi-year trend
SELECT DATE_TRUNC(order_date, YEAR) AS order_year, COUNT(*) AS order_count
FROM `partition_demo.orders_by_year`
GROUP BY order_year
ORDER BY order_year;
```

Note: EXTRACT(YEAR FROM order_date) = 2023 also works but prunes less reliably than a direct range/equality filter on the partitioning expression, since BigQuery's pruning logic works best on the exact column or expression the table is partitioned by.

Timestamp Partitioning — HOUR

What it does: partitions by `TIMESTAMP_TRUNC(order_ts, HOUR)` — the finest granularity BigQuery allows, and the first table here partitioned on a TIMESTAMP rather than a DATE column.

```
CREATE OR REPLACE TABLE `partition_demo.orders_by_hour`
PARTITION BY TIMESTAMP_TRUNC(order_ts, HOUR)
OPTIONS (partition_expiration_days = 400)
AS SELECT * FROM `partition_demo.orders_raw`;
```

Why HOUR, and why it needs an expiration policy

Some workloads genuinely need sub-day pruning — intraday operational dashboards, fraud monitoring, log analytics where queries filter to a specific hour or few hours. HOUR partitioning is how you get that.

The catch, and the teaching point: our data spans 4 years ≈ 35,000 hours, which exceeds the 4,000-partition limit per table. That's precisely why partition_expiration_days is set here — it's not an optional nicety, it's what keeps an HOUR-partitioned table with continuous ingestion under the partition-count ceiling by automatically dropping old partitions. This is a good moment to contrast with DAY/MONTH/YEAR, where the coarser grain meant retention wasn't forced by a hard limit.

Query

```
SELECT COUNT(*) AS order_count
FROM `partition_demo.orders_by_hour`
WHERE order_ts BETWEEN TIMESTAMP '2023-06-15 09:00:00'
                   AND TIMESTAMP '2023-06-15 12:00:00';
```

Ingestion-Time Partitioning (_PARTITIONTIME)

What it does: partitions purely by the time BigQuery loaded each row, using the pseudo-column `_PARTITIONTIME` — independent of any column in the data itself.

```
CREATE TABLE `partition_demo.orders_ingestion`
(
  order_id INT64,
  order_date DATE,
  order_ts TIMESTAMP,
  customer_id INT64,
  country STRING,
  category STRING,
  amount NUMERIC
)
PARTITION BY _PARTITIONTIME
OPTIONS (partition_expiration_days = 90);

-- Load a batch (simulates data arriving today)
INSERT INTO `partition_demo.orders_ingestion`
SELECT * FROM `partition_demo.orders_raw`
WHERE order_id BETWEEN 1 AND 100000;

-- Load into a specific historical partition via the bq CLI decorator
-- bq query --destination_table=partition_demo.orders_ingestion'$20240115' \
--   --use_legacy_sql=false \
--   'SELECT * FROM partition_demo.orders_raw WHERE order_id BETWEEN 100001 AND 200000'
```

Why this exists as a distinct case

Every table so far partitioned on a column that already existed in the source data (order_date, order_ts). Ingestion-time partitioning covers the opposite situation: source data with **no reliable date/timestamp column at all**, or a pipeline where you specifically want partitions to reflect "when did this land in BigQuery" rather than "what does the business event say." Streaming logs and raw landing-zone tables are the classic use case.

It's important to flag to learners that this decouples partition pruning from business-date filtering — a query filtering on order_date here gets no pruning benefit at all, only a filter on _PARTITIONTIME does. That's the trade-off: simplicity of setup (no dependency on a clean source column) versus a partition key that may not match what analysts actually query on.

Query

```
SELECT COUNT(*) AS order_count
FROM `partition_demo.orders_ingestion`
WHERE _PARTITIONTIME BETWEEN TIMESTAMP '2024-01-15' AND TIMESTAMP '2024-01-16';
```

Integer-Range Partitioning

What it does: partitions by fixed-width buckets of an INT64 column using `RANGE_BUCKET` — no date or timestamp involved at all.

```
CREATE OR REPLACE TABLE `partition_demo.orders_by_customer_range`
PARTITION BY RANGE_BUCKET(customer_id, GENERATE_ARRAY(0, 200000, 10000))
CLUSTER BY country
AS SELECT * FROM `partition_demo.orders_raw`;
```

Why integer-range, and when it's the right call

Not every table has a date to partition on. Multi-tenant systems partitioned by tenant_id, sharded customer tables, or numeric ID ranges tied to onboarding cohorts are all cases where the natural, frequently-filtered column is an integer, not a date. RANGE_BUCKET(column, GENERATE_ARRAY(start, end, interval)) is the mechanism — here customer_id 0-200,000 is split into 20 buckets of 10,000 each.

Query

```
SELECT COUNT(*) AS order_count
FROM `partition_demo.orders_by_customer_range`
WHERE customer_id BETWEEN 50000 AND 59999;
```

7 CLUSTERING

Clustering Only (No Partitioning)

What it does: physically sorts and co-locates rows into storage blocks ordered by the clustering columns — with no partitioning at all.

```
CREATE OR REPLACE TABLE `partition_demo.orders_clustered_only`
CLUSTER BY country, category, customer_id
AS SELECT * FROM `partition_demo.orders_raw`;
```

Why clustering-only makes sense on its own

Partitioning isn't always a fit: the table might be too small for partition overhead to pay off, or there may be no date/integer column that lines up with query patterns, or the useful filter columns are all low/medium-cardinality strings like country or category rather than something partitionable. Clustering-only covers exactly this gap — it delivers block-level pruning purely from sort order, without requiring a partitioning column at all.

Mechanically, clustering is **a sort, not an index**. BigQuery stores the data in blocks and keeps min/max metadata per block for each clustering column, in the order they're listed. A query filtering on the first clustering key (country) can skip whole blocks that don't contain matching values. A query filtering only on a later key (customer_id) benefits far less, because customer_id isn't sorted independently of country/category — it's only sorted within them.

Column order is not cosmetic — demonstrate the contrast

```
-- Prunes well: country is the leading (first) clustering key
SELECT SUM(amount) FROM `partition_demo.orders_clustered_only`
WHERE country = 'India';

-- Prunes far less: customer_id is the third/trailing clustering key
SELECT SUM(amount) FROM `partition_demo.orders_clustered_only`
WHERE customer_id = 42;
```

Compare bytes processed for these two queries in the console. The first should scan noticeably less data than the second, purely because of clustering column order — a useful, slightly counter-intuitive lesson for learners who assume all clustering columns are equally effective.

8 COMBINED

Partitioning + Clustering Together

What it does: combines both mechanisms — partitioning narrows which partitions (e.g. days) are read at all, and clustering then sorts *within* each partition so BigQuery can skip blocks inside the surviving partitions too.

```
CREATE OR REPLACE TABLE `partition_demo.orders_by_day`
PARTITION BY order_date
CLUSTER BY country, category
AS SELECT * FROM `partition_demo.orders_raw`;

-- Same table used in Section 1 — revisit it here as the capstone example
```

Why combine them — this is the realistic production pattern

Partitioning and clustering solve different problems and stack cleanly: partitioning eliminates entire partitions (coarse-grained pruning, typically by date), clustering then eliminates blocks within whatever partitions remain (fine-grained pruning, typically by dimension columns). Most production BigQuery tables use both together, because most real query patterns filter on a date range *and* one or more dimension columns at the same time — "last month's orders from India," not just "last month's orders" or just "orders from India."

Layered pruning, demonstrated in three steps

```
-- Step 1: no pruning at all — full table scan
SELECT country, SUM(amount) AS revenue
FROM `partition_demo.orders_raw`
WHERE order_date BETWEEN '2023-06-01' AND '2023-06-30'
GROUP BY country;

-- Step 2: partition pruning only — scans ~30 daily partitions instead of all ~1,460
SELECT country, SUM(amount) AS revenue
FROM `partition_demo.orders_by_day`
WHERE order_date BETWEEN '2023-06-01' AND '2023-06-30'
GROUP BY country;

-- Step 3: partition + cluster pruning — scans ~30 partitions, then skips
-- non-India blocks within each one
SELECT SUM(amount) AS revenue
FROM `partition_demo.orders_by_day`
WHERE order_date BETWEEN '2023-06-01' AND '2023-06-30'
  AND country = 'India';
```

Run all three in sequence and read off "bytes to be processed" from the console before each run: Step 1 should show the full table size, Step 2 a small fraction of it (roughly days-filtered / total-days), and Step 3 smaller still. That progression is the single clearest way to make the difference between the two mechanisms visible, rather than just described.

Enforcing the discipline: require_partition_filter

```
ALTER TABLE `partition_demo.orders_by_day`
SET OPTIONS (require_partition_filter = true);
```

A good closing point: this option makes BigQuery reject any query on the table that doesn't include a filter on the partitioning column, which is a simple, effective guardrail against accidental full-table scans on a large production table once it's partitioned.

Reference: Inspecting Partitions & Clustering Metadata

```
-- Partition-level row/byte counts
SELECT partition_id, total_rows, total_logical_bytes
FROM `partition_demo.INFORMATION_SCHEMA.PARTITIONS`
WHERE table_name = 'orders_by_day'
ORDER BY partition_id;

-- Which columns are clustering keys, and in what order
SELECT table_name, column_name, clustering_ordinal_position
FROM `partition_demo.INFORMATION_SCHEMA.COLUMNS`
WHERE clustering_ordinal_position IS NOT NULL
ORDER BY table_name, clustering_ordinal_position;
```